

23CSE352: Big Data Analytics

Chronological Execution Log

Commands Executed From Starting

Team Members:

Sheela Akshar Sakhi

Nishanth S Gowda

August 14, 2026

Contents

1	Overview	2
2	Phase 1: Project Repository Setup & Git Initialization	2
3	Phase 2: Virtual Machine Environment Setup	2
4	Phase 3: Hadoop & YARN Daemon Startup	2
5	Phase 4: Data Download & Preprocessing	3
6	Phase 5: HDFS Storage Upload	3
7	Phase 6: MapReduce Compilation & YARN Execution	3
8	Phase 7: Apache Hive Metastore Reset & SQL Analytics	4
9	Phase 8: Query Output Extraction for Screenshots	5
10	Phase 9: LaTeX Compilation & Git Operations	5

1 Overview

This document provides a complete, step-by-step chronological log of every single terminal command executed to build, run, evaluate, and generate deliverables for the Big Data Analytics of Crime Patterns project.

2 Phase 1: Project Repository Setup & Git Initialization

Ran on Host Machine (Mac Terminal):

```
1 # Navigate to workspace directory
2 cd /Users/aksharsakhi/Documents/Files/VScode/Amrita/Big_Data
3
4 # Create and rename project directory
5 mv BigDataProject Big-Data-Analytics-of-Crime-Patterns
6 cd Big-Data-Analytics-of-Crime-Patterns
7
8 # Initialize Git repository and add remote
9 git init
10 git remote add origin https://github.com/aksharsakhi/Big-Data-Analytics-
    -of-Crime-Patterns.git
```

3 Phase 2: Virtual Machine Environment Setup

Ran on Virtual Machine Terminal (hadoop@aksharsakhi-QEMU-Virtual-Machine):

```
1 # Clone the repository onto the VM
2 git clone https://github.com/aksharsakhi/Big-Data-Analytics-of-Crime-
    Patterns.git
3
4 # Enter project folder
5 cd Big-Data-Analytics-of-Crime-Patterns
6
7 # Grant execution permissions to shell scripts
8 chmod +x *.sh
```

4 Phase 3: Hadoop & YARN Daemon Startup

Ran on VM Terminal to manually start Hadoop services (without using .sh extensions):

```
1 # Start HDFS Storage Daemons
2 hdfs --daemon start namenode
3 hdfs --daemon start datanode
4 hdfs --daemon start secondarynamenode
5
6 # Start YARN Resource Manager & Node Manager Daemons
7 yarn --daemon start resourcemanager
8 yarn --daemon start nodemanager
9
10 # Verify running Java daemons
11 jps
```

5 Phase 4: Data Download & Preprocessing

Ran on VM Terminal:

```
1 # Pull latest repository updates
2 git pull
3
4 # Download Chicago Crimes dataset (10,000 records via Socrata API)
5 mkdir -p dataset
6 wget -q -O dataset/chicago_crimes_10k.csv "https://data.cityofchicago.org/resource/ijzp-q8t2.csv?&$limit=10000"
7
8 # Clean multi-line string newlines using Python
9 python3 preprocess.py
```

6 Phase 5: HDFS Storage Upload

Ran on VM Terminal:

```
1 # Create HDFS destination directory
2 hdfs dfs -mkdir -p /user/hadoop/bigdata_project/crimes
3
4 # Upload cleaned dataset to HDFS datalake
5 hdfs dfs -put -f dataset/chicago_crimes_clean.csv /user/hadoop/
  bigdata_project/crimes/
6
7 # Verify HDFS upload (Screenshot: hdfs_screenshot.png)
8 hdfs dfs -ls /user/hadoop/bigdata_project/crimes/
```

7 Phase 6: MapReduce Compilation & YARN Execution

Ran on VM Terminal:

```
1 # Clean previous build artifacts
2 rm -rf classes && mkdir classes
3
4 # Compile Java MapReduce program targeting Java 8 runtime
5 javac -source 1.8 -target 1.8 -classpath 'hadoop classpath' -d classes
  src/CrimeTypeCount.java
6
7 # Package compiled class files into JAR executable
8 jar -cvf crimecount.jar -C classes/ .
9
10 # Remove stale HDFS output directory if present
11 hdfs dfs -rm -r -f /user/hadoop/bigdata_project/crime_output
12
13 # Execute MapReduce job on Hadoop YARN Cluster (Screenshot:
  mr_execution.png)
14 hadoop jar crimecount.jar bigdata.CrimeTypeCount /user/hadoop/
  bigdata_project/crimes /user/hadoop/bigdata_project/crime_output
15
16 # Display MapReduce frequency counts output (Screenshot: mr_output.png)
17 hdfs dfs -cat /user/hadoop/bigdata_project/crime_output/part-r-00000 |
  head -n 20
```

8 Phase 7: Apache Hive Metastore Reset & SQL Analytics

Ran on VM Terminal:

```
1 # Reset Derby metastore to prevent locks
2 rm -rf metastore_db
3
4 # Initialize Derby Metastore database schema
5 schematool -dbType derby -initSchema
6
7 # Batch execution of all 10 SQL queries
8 hive -f crime_analysis.hql > hive_output.txt
9
10 # Interactive Hive execution
11 hive
```

Interactive Hive Queries executed inside hive> shell:

```
1 USE default;
2
3 -- Query 1: SELECT
4 SELECT id, primary_type, description, crime_date FROM crimes LIMIT 10;
5
6 -- Query 2: WHERE
7 SELECT id, primary_type, description, arrest FROM crimes WHERE
8     primary_type = 'NARCOTICS' AND arrest = 'true' LIMIT 10;
9
10 -- Query 3: ORDER BY
11 SELECT case_number, primary_type, crime_date FROM crimes WHERE
12     primary_type = 'HOMICIDE' ORDER BY crime_date DESC LIMIT 10;
13
14 -- Query 4: GROUP BY & COUNT
15 SELECT location_description, COUNT(*) AS crime_count FROM crimes GROUP
16     BY location_description ORDER BY crime_count DESC LIMIT 15;
17
18 -- Query 5: HAVING
19 SELECT primary_type, COUNT(*) as total FROM crimes GROUP BY
20     primary_type HAVING COUNT(*) > 500 ORDER BY total DESC;
21
22 -- Query 6: SUM
23 SELECT district, SUM(CASE WHEN arrest = 'true' THEN 1 ELSE 0 END) AS
24     total_arrests FROM crimes WHERE district IS NOT NULL AND district !=
25     '' GROUP BY district ORDER BY total_arrests DESC LIMIT 10;
26
27 -- Query 7: AVG
28 SELECT beat, AVG(CASE WHEN domestic = 'true' THEN 1.0 ELSE 0.0 END) AS
29     avg_domestic FROM crimes WHERE beat IS NOT NULL AND beat != '' GROUP
30     BY beat ORDER BY avg_domestic DESC LIMIT 10;
31
32 -- Query 8: MAX
33 SELECT district, COUNT(*) AS district_total FROM crimes WHERE district
34     IS NOT NULL AND district != '' GROUP BY district ORDER BY
35     district_total DESC LIMIT 1;
36
37 -- Query 9: MIN
38 SELECT district, COUNT(*) AS district_total FROM crimes WHERE district
39     IS NOT NULL AND district != '' GROUP BY district ORDER BY
40     district_total ASC LIMIT 1;
```

```

29
30 -- Query 10: JOIN
31 SELECT c.case_number, c.primary_type, a.area_name FROM crimes c JOIN
    community_areas a ON (c.community_area = a.area_code) LIMIT 15;
32
33 exit;

```

9 Phase 8: Query Output Extraction for Screenshots

Ran on VM Terminal:

```

1 # Query 1 to Q3 Output
2 head -n 35 hive_output.txt
3
4 # Query 4 to Q7 Output
5 sed -n '36,75p' hive_output.txt
6
7 # Query 8 to Q10 Output
8 tail -n 35 hive_output.txt

```

10 Phase 9: LaTeX Compilation & Git Operations

Ran on Host System (Mac Terminal):

```

1 # Change to presentation directory
2 cd presentation
3
4 # Compile LaTeX documents
5 pdflatex -interaction=nonstopmode report.tex
6 pdflatex -interaction=nonstopmode presentation.tex
7 pdflatex -interaction=nonstopmode commands_executed.tex
8
9 # Clean up auxiliary build files
10 find . -type f \( -name "*.aux" -o -name "*.log" -o -name "*.nav" -o -
    name "*.out" -o -name "*.snm" -o -name "*.toc" -o -name "*.vrb" \) -
    delete
11
12 # Commit and push to GitHub
13 git add .
14 git commit -m "Add commands_executed.pdf generated from LaTeX"
15 git push

```